

Exchange Development Introduction Document

C++ Exchange: Trading Competition: Fall 2026

Trading at Georgia Tech

Quantitative Development

1 Introduction

1.1 Document Goals

This document covers the C++ and Drogon material needed to work on the exchange for the Georgia Tech trading competition. You should already be comfortable with basic C++ syntax. The design principles, Drogon concepts, and project-specific material will be covered and applied over the next one to two weeks.

There is a short C++ refresher for anyone who needs it. Skip it if you are already comfortable with the material. The software design and Drogon sections are the main assigned readings.

1.2 Stack Overview

The exchange will be written in **C++20**. **Drogon** will handle the HTTP and WebSocket layer. The matching engine and order book should stay as normal C++ code so they can be built and tested without starting a Drogon server.

The project will roughly be split into:

participant client $\longleftrightarrow$ Drogon API $\longleftrightarrow$ exchange core $\longrightarrow$ market data

1.3 Reading Philosophy

For each concept, make sure you understand **why it exists, how it works, and when you would use it**. Try things as you go. Write a small example, change one of the provided examples, or use the concept in the warm-up project.

There is no need to memorize every API or minor detail. You should be able to use the main ideas and explain the decisions you make with them. The readings will be much more useful if you actually write code alongside them.

2 Required Readings

2.1 C++ Refresher: Skippable

If you are already comfortable writing modern C++, skip this section. Otherwise, use *A Tour of C++, 3rd Edition* as a quick refresher:

- **Chapter 3: Modularity.** Separate compilation, namespaces, and function interfaces.
- **Chapter 4: Error Handling.** Exceptions, invariants, alternatives, and assertions.
- **Chapter 5: Classes.** Concrete types, abstract types, virtual functions, and class hierarchies.
- **Chapter 6: Essential Operations.** Copy/move semantics and resource management.
- **Chapter 12: Containers.** Focus on vector, map, and unordered_map.

- **Chapter 15: Pointers and Containers.** Ownership, `unique_ptr`, `shared_ptr`, `span`, `optional`, and `variant`.
- **Chapter 18: Concurrency.** Threads, shared data, synchronization, communication, and coroutines.

Book page: stroustrup.com/tour3.html.

Free alternative: LearnCpp. Review the equivalent material on move semantics, smart pointers, standard containers, lambdas, virtual functions, and multithreading. Start at learncpp.com and skip anything you already know.

2.2 C++ Software Design: Required

Read the following sections from Klaus Iglberger's *C++ Software Design*:

- **Chapter 1: The Art of Software Design.** Read the full chapter. For this project, focus on separation of concerns, avoiding unnecessary coupling, testability, and designing code that can change without requiring large rewrites.
- **Chapter 2: The Art of Building Abstractions.** Read the full chapter. Pay particular attention to ownership of abstractions, dependency inversion, and architectural boundaries.
- **Chapter 3: The Purpose of Design Patterns.** Useful if you are not familiar with common design-pattern terminology.
- **Chapter 5: Strategy and Command.** Read Guidelines 19, 20, 22, and 23. Strategy, composition, and value semantics will be useful for exchange policies and domain objects. Guideline 21 on Command is optional for now.
- **Chapter 6: Adapter, Observer, and CRTP.** Read Guidelines 24 and 25 only. Adapter is useful at protocol and framework boundaries. Observer is useful for market-data and event publication. Skip CRTP for now.
- **Chapter 7, Guidelines 28–29: Bridge/Pimpl.** Useful if public headers or physical dependencies start causing build-time or API-boundary problems.
- **Chapter 8: Type Erasure.** Useful if the exchange eventually needs runtime-pluggable policies without relying on large inheritance hierarchies.
- **Chapter 10: Singleton.** Read this before adding global services or global exchange state. The main concern here is the effect globals have on dependencies and testing.

Book page: *C++ Software Design* by Klaus Iglberger.

2.3 Drogon Documentation: Required

Read these in order:

1. **Quick Start.** Drogon Wiki: ENG-03 Quick Start
2. **Controller Introduction.** Drogon Wiki: ENG-04-0 Controller Introduction
3. **HttpController.** Drogon Wiki: ENG-04-2 HttpController
4. **WebSocketController.** Drogon Wiki: ENG-04-3 WebSocketController
5. **Threading Model.** Drogon Wiki: Understanding Drogon's Threading Model
6. **Configuration.** Drogon Wiki: ENG-11 Configuration File

7. **Testing.** Drogon Wiki: ENG-19 Testing Framework
8. **Coroutines.** Read this if a component performs asynchronous database, HTTP-client, Redis, or other awaitable I/O. Coroutines can make control flow cleaner, but they do not make blocking work or shared state automatically safe.
9. **Middleware and Filters.** Useful once authentication, participant validation, rate limits, or common request handling are added.
10. **Plugins.** Useful if long-lived services are initialized and connected through Drogon's plugin system.
11. **Database / ORM.** Only needed if competition state, fills, or replay data need to be persisted.
12. **Benchmarks.** Useful before performance testing. Test the exchange path separately from the framework itself.

Pay particular attention to the threading-model page. You should understand why blocking an event-loop thread is a problem, why controller state can be accessed concurrently, and why asynchronous code can still have data races.

2.4 Drogon Repository: Required

Look through these examples in the Drogon repository:

- `examples/websocket_server`: WebSocket connection lifecycle, per-connection context, and Drogon's PubSubService. The publish/subscribe setup is relevant to market-data fanout.
- `examples/jsonstore`: A concurrent in-memory REST service. Pay attention to how shared state is handled behind HTTP handlers.

Use these examples to learn Drogon's conventions. Do not copy their structure directly into the exchange. Matching and order-book logic should stay outside the controller layer.

3 Warm-up Project

3.1 Description

Build a **single-symbol in-memory limit-order-book service** using C++20 and Drogon. For this project, an *instrument* is simply the tradable symbol or ticker whose orders are stored in the book.

Your service should:

1. Accept a limit order containing side, price, and quantity through an HTTP endpoint.
2. Assign each accepted order a unique order ID.
3. Match orders using **price-time priority**: match the best available price first. For orders at the same price, match the oldest order first.
4. Support cancellation of a resting order by order ID.
5. Return at least the best five bid and ask price levels through an HTTP endpoint.
6. Publish a WebSocket update whenever the visible book changes or a trade occurs.

You do not need authentication, persistence, multiple instruments, or advanced order types for the warm-up.

3.2 Requirements

- The matching engine and order-book code must have no Drogon types in its public interface. It should be possible to unit test it without running a server.
- Controllers should convert network messages into exchange-domain values, call the exchange core, and convert the results back into HTTP or WebSocket messages.
- Shared mutable state must have a clear concurrency strategy. A simple correct solution is completely fine for the warm-up.
- Do not block Drogon's event-loop threads with sleeps, blocking network calls, or long-running work.
- Use RAI and standard ownership types. Raw owning `new/delete` should not be necessary.
- Include tests for price priority, FIFO within a price, partial fills, crossing multiple resting orders, cancellation, and invalid cancellation.
- Use CMake and compile with warnings enabled.

3.3 Suggested API

Method	Path	Purpose
POST	/orders	Submit an order
DELETE	/orders/{id}	Cancel an order
GET	/book	Read a book snapshot
GET (WS)	/marketdata	Stream book/trade updates

3.4 Submission

Due date: Friday, September 25, 2026.

Publish the final version to GitHub and submit the repository link through the channel specified by the project leads.

Submit a short video showing the application in Postman. Demonstrate the endpoints with valid requests, show at least one invalid request being rejected with an appropriate error response, and include the WebSocket market-data endpoint.

Be ready to explain one decision involving ownership, one involving concurrency, and one involving the boundary between Drogon and the exchange core.